File Encoding — An Engineering Walkthrough

The same material as the academic monograph `EncoderGuide.Academic.en.md`, minus formalism for its own sake: shorter notation, more "why it is built this way" and "how it behaves on real data". Section numbering matches in both versions, so the cross-references are interchangeable. The from-scratch tutorial is `EncoderGuide.Tutorial.en.md`; format context is in `AlgorithmGuide.en.md` (sections 2–4). The reverse process is `DecoderGuide.Engineer.en.md`.

1. What we are building

`FileEncoder` turns a file into a list of 75-byte packets: `T` data sectors (N data + M ECC) plus `H` copies of the header. Two constructor parameters: `eccPercent` (default 10) and `headerPercent` (default 3). Everything else is derived. API: `Encode` (bytes or a `Stream`), `EncodeToText` (Base64, one line per packet), `EncodeWithStats` (+ an `EncodeStats` summary). Progress is global 0–100; cancellation via `CancellationToken`.

2. The loss model and redundancy

The stream survives two kinds of damage: lost chunks (healed by ECC and header repeats) and garbage/desynchronization (healed by the decoder's sliding window). Hence the two percents:

- **ECC** — how many data volumes can be lost irrecoverably: `M = max(1, ⌈N·p/100⌉)`. The loss of any `k ≤ M_avail` data volumes is recoverable; lost ECC volumes cost nothing.
- **Headers** — the survivability of file identification: `H = max(3, ⌈T·h/100⌉)`. Always three copies: start, end, middle.

On small files the percents act as "at least one/three" — the redundancy is multiples of the whole, but that is the price of the guarantee.

3. Format constants

Constant	Value	Meaning
`PacketSize`	75	the indivisible packet
`PayloadSize`	64	D2
`HeaderContentSize`	51	H1–H4
`HeaderHashSize`	24	H5, 192 bits
`SectorHashSize`	9	D3, 72 bits
`MaxFileSizeField`	16,777,215	the file ceiling (3-byte field)
`MaxDataVolumes`	65,535	the N+M ceiling (GF(2^{16}))

Exceeding them throws `InvalidOperationException` — never a silently truncated output.

4. Data preparation

Order: checks → `FileNameCodec.Pack(Path.GetFileName(name))` → `Sha256Compact.HashData(content)` → dimensions:

```
N = max(1, ⌈FileSize/64⌉);  M = max(1, ⌈N·ecc%/100⌉) or 0;  T = N+M
```

Slicing: N payloads of 64 bytes, the last one zero-padded. Intermediate buffers are wiped after the packets are assembled: the file's contents must not linger in memory longer than necessary.

The name in 14 bytes: split at the first dot, the extension (including `.tar.gz`) is preserved whole, the base is quantized, a `~` is appended on truncation. A base budget < 2 bytes — refusal. Failing at name-packing time is right: better an exception now than a header with an empty/mangled name.

5. RS encoding

`RsCodecAdapter.Encode(dataPayloads, M)`: a 64-byte volume = 32 GF(2^{16}) symbols (UInt16 LE); for each of the 32 positions a column of N data symbols is assembled, and `RsRaid16.Process` appends M ECC symbols — linear combinations over the field. Positions are independent; progress ticks per symbol. The `K+M ≤ 65535` check runs before the start. The field math itself lives in the reference `RsRaid16` and is untouched.
Output properties: any set of `M` available ECC covers `M` lost data volumes; recovery is exact linear algebra — no search, no probabilities.

6. The header packet

`HeaderContent` serializes into 51 bytes (H1 name, H2 size LE, H3 SHA-256, H4 M LE), then `H5 = Trunc24(SHA-256(51 bytes))` — and the packet is ready: 51+24. H5's double role: an autonomous self-check for the header (the decoder needs to know nothing in advance) and the seed for the D3 of all the file's sectors. Truncation to 192 bits suffices: a false positive ≈ 2^{-192}.

7. Data sectors

Each payload → a sector `D1(2 LE) ‖ D2(64) ‖ D3(9)`, where `D3 = Trunc9(SHA-256(H5 ‖ D1 ‖ D2))`. The hash input is 90 bytes. The key invariant: **a sector cannot be verified without H5**, so foreign packets do not stick to the file, and its own sectors are recognized only after the header (the "sector before header" problem is solved on the decoder side by targeted rebinding — `DecoderGuide.Engineer.en.md`, section 10).

8. Header placement

`ArrangePackets(headerPacket, sectors, H)`: the first and last packets are headers; the intermediate `H-2` copies are inserted after every `⌈T/(H-1)⌉`-th sector. Evenness means: cutting out any single interval of the stream leaves ≥ 2 copies with probability close to one at realistic T. Sector order is natural: no shuffling is needed — the decoder's windows survive desynchronization on their own.

9. Output formats and PacketIO

- **Base64** (`.DataShield.txt`): a line = a packet = exactly 100 characters; `=` never occurs (75 is a multiple of 3). The decorative frames `>[name][size][SHA-256:hex]` / `<[…]` are for humans; the decoder's filter discards them.
- **Binary** (`.DataShield.bin`): raw packets back to back.

`PacketIO.WriteFile` writes a file or a stream; `GetDefaultOutputPath` = input + the format's extension; `DetectFormat` on reading looks at the extension (anything unrecognized → Base64).

10. Progress and cancellation

Global-scale phases: preparation 0–10, ECC 10–75 (rescaled by `ScaledProgress`), packetization 75–100, finale `Done`. Whole-percent throttling (`ProgressThrottle`). Cancellation is checked between phases and inside the RS symbol loop; the exception goes outward; a half-stream is never returned.

11. Statistics and wipes

`EncodeWithStats` returns `EncodeStats(FileSize, Sha256, DataCount, EccCount, TotalPackets, HeaderCopies)` — everything the UI needs to report "what you will get". After `Encode`, the data payloads, ECC payloads, and header bytes are zeroed. The resulting packets live on as is — their contents inevitably contain the data.

12. Stream input

`ReadStreamContent`: a seekable stream is read into an array by the remainder (`Length - Position`, ceiling checked); a non-seekable one via a `MemoryStream` copy. The stream is not closed; any `Stream` works, including `MemoryStream`. The full read is not a whim: SHA-256 and RS encoding require the entire contents.

13. End-to-end example

A 1000-byte file, 10% ECC, 3% headers: `N=16`, `M=2`, `T=18`, header copies `max(3, ⌈0.54⌉)=3`, interval 9 → the stream `H D0…D8 H D9…D17 H`, 20 packets: 1500 bytes binary or 20×100 Base64 characters. The last data volume: 40 bytes of data + 24 zeros. For large N the expansion converges to

≈132% (binary) and ≈176% (Base64).

14. Parameters and limits

Parameter	Range	Effect
`eccPercent`	0…100 (beyond that it hits the GF ceiling)	M volumes; 0 = no recovery
`headerPercent`	0…100	H copies; always ≥ 3
File size	≤ 16,777,215	field H2
`N+M`	≤ 65,535	GF(2^{16}) field

Rule of thumb: `eccPercent=10` loses data irrecoverably only when > 10% of data volumes **and** the corresponding ECC are lost; `headerPercent=3` puts a header copy roughly every 33 sectors.

15. Edge cases

- Empty file → `N=1` (one zero volume), assembles correctly.
- `eccPercent=0` → the ECC phase is skipped entirely (zero overhead).
- A 1-byte file with ECC → `N=1, M=1, T=2`, 6400% redundancy — the guarantee works.
- A long extension (base < 2 bytes) → refusal at name packing.
- A file beyond the ceilings → explicit exceptions with the numbers in the message.

16. What is guaranteed

1. Either a complete, correct stream or an exception — no intermediate states.
2. `H3` is the exact SHA-256; the assembly arbitration is impeccable.
3. Every sector is bound by the `H5` seed; substitution fails the D3 check.
4. The loss of ≤ M data volumes is recoverable given surviving ECC (exact algebra, not a heuristic).
5. A minimum of 3 header copies, evenly: start/middle/end.
6. Intermediate buffers are wiped: no extra copies of the contents linger in memory.